TEMA: Generación de CRUD para tablas relacionadas

Resultados de Aprendizaje:

- Que los estudiantes comprendan como generar modelos con relaciones foráneas a otras tablas para las generaciones de aplicaciones robustas
- Que los estudiantes comprendan los pasos necesarios para la realización de carga de imanes al servidor

DESARROLLO

Cuando tabjamos con lagun proyecto tenemos la necesitada de poder cargar archivos o imagens , en esta pracrica aprendemroes a como getiones esta necedidas trabajando dede el crud de la tabla de productos

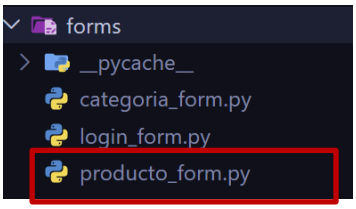
Requerimientos:

Para la realizacion de esta guia usted debe de contar con los siguientes elementos

- Contar con el proyecto comercial generado en la guia anterios
- Contar con un servidor de base de datos mysql

creacion de formularios

1. Para poder eliminar o crear datos de productos para nuestra aplicasion utilizaremos los formularios que nos permite dijando realziar para ello deberemos de crear el archivo **catalogo/forms/producto_form.py** como se muestra en la imagen siguiente



2. Editamos el arcivo **catalogo/forms/producto_form.py** quedando de la siguiente manera

```python
from django import forms
from django.utils import timezone
from django.core.validators import FileExtensionValidator
from ..models.producto import Producto
from ..models.categoria import Categoria


class ProductoForm(forms.Form):
    nombre = forms.CharField(max_length=100, label="Nombre")
    descripcion = forms.CharField(
        max_length=255, required=False,
        label="Descripción",
        widget=forms.Textarea(attrs={'rows': 3}))
    precio_compra = forms.DecimalField(
        max_digits=10, decimal_places=2, label="Precio de compra", min_value=0)
    precio_venta = forms.DecimalField(
        max_digits=10, decimal_places=2, label="Precio de venta", min_value=0)
    stock = forms.IntegerField(label="Stock", min_value=0)
    # generación de llave foránea y carga de datos de la tabla de categoria
    categoria = forms.ModelChoiceField(
        queryset=Categoria.objects.all(), label="Categoría", empty_label="Seleccione una categoría")

    # carga de imagen y validaciones de tipo
    imagen = forms.ImageField(required=False, label="Imagen", validators=[FileExtensionValidator(
        allowed_extensions=["jpg", "png"])])

    def clean(self):
        cleaned_data = super().clean()
        precio_compra = cleaned_data.get('precio_compra')
        precio_venta = cleaned_data.get('precio_venta')
        if precio_compra is not None and precio_venta is not None:
            if precio_venta <= precio_compra:
                self.add_error(
                    'precio_venta',
                    'El precio de venta debe ser mayor que el precio de compra.',
                )

        return cleaned_data

    def save(self):
        Producto.objects.create(
            nombre=self.cleaned_data['nombre'],
            descripcion=self.cleaned_data['descripcion'],
            precio_compra=self.cleaned_data['precio_compra'],
            precio_venta=self.cleaned_data['precio_venta'],
            stock=self.cleaned_data['stock'],
            categoria=self.cleaned_data['categoria'],
            imagen=self.cleaned_data.get('imagen', None),
        )

```

```python
    def update(self, producto):
        imagen_nueva = self.cleaned_data.get('imagen')
        imagen_nombre_anterior = producto.imagen.name
        storage = producto.imagen.storage

        producto.nombre = self.cleaned_data['nombre']
        producto.descripcion = self.cleaned_data['descripcion']
        producto.precio_compra = self.cleaned_data['precio_compra']
        producto.precio_venta = self.cleaned_data['precio_venta']
        producto.stock = self.cleaned_data['stock']
        producto.categoria = self.cleaned_data['categoria']
        # si la imagen es enviada la actualiza porque la imagen es no obligatoria de enviar
        if imagen_nueva:
            producto.imagen = imagen_nueva
        producto.save()

        #  valida  cuando eliminar la imagen
        if (
            imagen_nueva
            and imagen_nombre_anterior
            and imagen_nombre_anterior != producto.imagen.name
        ):
            # elimina la imagen del servidor
            storage.delete(imagen_nombre_anterior)

```

Expliasion de codigo:

```python
 categoria = forms.ModelChoiceField(
        queryset=Categoria.objects.all(), label="Categoría",
empty_label="Seleccione una categoría")
```

nos permite relacionar los campos de un formulario con de producto con la categoria y llena los datos utilizando el modelo Categoria

```python
   imagen = forms.ImageField(required=False, label="Imagen",
validators=[FileExtensionValidator(
        allowed_extensions=["jpg", "png"])])
```

permite agegar un componete especializado para realizar la carga de imágenes desde el formulario y nos pretme valiad que la extenciones de los archivos permitidos

```python
 imagen_nombre_anterior = producto.imagen.name
```

permite acceder el nombre de la imagen si esta existe , si no existe regresa null

```python
storage = producto.imagen.storage
```

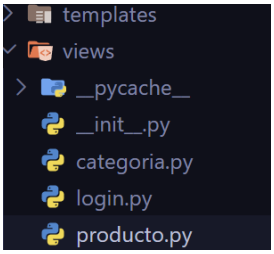
permite acceder al sistema de lamacenamiento

```python
 storage.delete(imagen_nombre_anterior)
```

permite elimiar la imagen el servidor

Genracion de vistas para el CRUD de productos

3. Para poder realizar el crud dr producto debera de crear la vistas correspondiente para esto crearemos el archivo **catalogo/views/producto.py ,** como se muestra en la imagen



4. Para continuar con el crud editamos nuesta vista **catalogo/views/producto.py ,** no se precupe por los template que aun no los posee los crearemos en el siguiente paso

```python
from django.shortcuts import render, redirect, get_object_or_404
from ..models.producto import Producto
from ..forms.producto_form import ProductoForm
# Create your views here

def lista_productos(request):
    lista_productos = Producto.objects.all()
    return render(request, "producto/lista.html", context={
        'productos': lista_productos
    })
```

```python

def delete_producto(request, id):
    producto = get_object_or_404(Producto, id=id)
    if producto.imagen:
        producto.imagen.storage.delete(producto.imagen.name)
    producto.delete()
    return redirect('list_producto')


def add_producto(request):
    if request.method == 'POST':
        form = ProductoForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
            return redirect('list_producto')
        else:
            return render(request, "producto/add.html", context={
                'form': form
            })
    return render(request, "producto/add.html", context={
        'form': ProductoForm()
    })
```

```python

def update_producto(request, id):
    producto = get_object_or_404(Producto, id=id)
    if request.method == 'POST':
        form = ProductoForm(request.POST, request.FILES)
        if form.is_valid():
            form.update(producto)
            return redirect('list_producto')
        else:
            return render(request, "producto/update.html", context={
                'form': form
            })

    form = ProductoForm(initial={
        'nombre': producto.nombre,
        'descripcion': producto.descripcion,
        'precio_compra': producto.precio_compra,
        'precio_venta': producto.precio_venta,
        'stock': producto.stock,
        'categoria': producto.categoria,
        'imagen': producto.imagen
    })
    return render(request, "producto/update.html", context={
        'form': form,
        'producto': producto
    })
```

Explicasion de codigo

- La funcon lista producto nos permitra visualizar todos los producto que esten en la base de datos

```python
def lista_productos(request):
    lista_productos = Producto.objects.all()
    return render(request, "producto/lista.html", context={
        'productos': lista_productos
    })
```

- Productos add nos permirira agregar un producto a la base de datos , pero tiene que tener en cueenque esta vista nos permitira urilizarla con el metodo get , donde solo enviar al template add.html con un formulario a mostrar , pero cuanod realice el post , cargara el formulario con todos los datos enviado por post , y podremos valiar los datros , pero lo mas interesenta es que este formulario recibe un parametro por request llamdo FILE que son los archivo que envie el usuario y si los datos no son correctos son redirigidos al mimos template con el formulario que tiene los datos de error para ser rendirizado

```python
def add_producto(request):
    if request.method == 'POST':
        form = ProductoForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
            return redirect('list_producto')
        else:
            return render(request, "producto/add.html", context={
                'form': form
            })
    return render(request, "producto/add.html", context={
        'form': ProductoForm()
    })
```

- Este foncion no permite eliminar el producto , pero tenga en cuenta que si desea eliminar un producto debera de preguntar si tiene imagen , si es asi

```python
def delete_producto(request, id):
    producto = get_object_or_404(Producto, id=id)
    if producto.imagen:
        producto.imagen.storage.delete(producto.imagen.name)
    producto.delete()
    return redirect('list_producto')
```

- En la funcion de actualizar nuevamente podremos utilizala cuanod se realizar un get para visualizar el formulario con el id correspondiente y con el post para actualizar

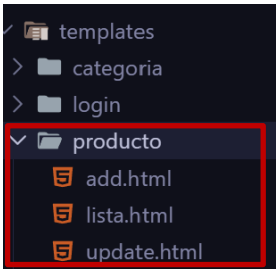
```python

def update_producto(request, id):
    producto = get_object_or_404(Producto, id=id)
    if request.method == 'POST':
        form = ProductoForm(request.POST, request.FILES)
        if form.is_valid():
            form.update(producto)
            return redirect('list_producto')
        else:
            return render(request, "producto/update.html", context={
                'form': form
            })

    form = ProductoForm(initial={
        'nombre': producto.nombre,
        'descripcion': producto.descripcion,
        'precio_compra': producto.precio_compra,
        'precio_venta': producto.precio_venta,
        'stock': producto.stock,
        'categoria': producto.categoria,
        'imagen': producto.imagen
    })
    return render(request, "producto/update.html", context={
        'form': form,
        'producto': producto
    })
```

Creacion de template

5. Para la creacion de los template realacionado al crud de productos deberemos de crer el siguiente sitema de directorios



6. Editamos el template **catagolo/templates/producto/lista.html** de la siguiente manera

```
{% extends "base.html" %}
{% block titulo %} Productos{% endblock titulo %}

{% block contenido %}
  <h1 class="font-bold text-balance">Lista de Categorias</h1>
<a href="{% url "add_producto" %}" class="p-2 bg-blue-600 my-4 inline-block text-white rounded hover:bg-blue-700">Agregar Producto</a>
<div class="grid grid-cols-7 gap-2 border-b max-w-4xl font-bold">
  <p>Nombre</p>
  <p>Precio Venta</p>
  <p>Precio Compra</p>
  <p>Stock</p>
  <p>Image</p>
  <p>Eliminar</p>
  <p>Actualizar</p>
</div>
{% if productos %}
{% for producto in productos %}
<div class="grid grid-cols-7 gap-2 max-w-4xl hover:bg-gray-300 items-center ">
  <p>{{producto.nombre}}</p>
  <p>{{producto.precio_compra}}</p>
  <p>{{producto.precio_venta}}</p>
  <p>{{producto.stock}}</p>
  {% if producto.imagen %}
    <img src="{{producto.imagen.url}}" alt="{{producto.nombre}}" class="w-16 h-16 shadow"></img>
  {% else %}
    <p> no hay imagen disponible</p>
  {% endif %}
  <a href="{% url 'delete_producto' producto.id %}" class="cursor-pointer hover:text-blue-600 ">Eliminar</a>
  <a href="{% url 'update_producto' producto.id %}" class="cursor-pointer hover:text-blue-600 ">Actualizar</a>
</div>
{% endfor %}
{% else %}
  <p>No hay productos registrados </p>
{% endif %}
{% endblock contenido %}

```

7. Editamos el template **catagolo/templates/producto/add.html ,** muy importante no olvidar el enctype="multipart/form-data" , para poder realizar la carga de la imagen

```
{% extends "base.html" %}
{% load tailwind_filters %}
{% block titulo %} agregar Producto{% endblock titulo %}
{% block contenido %}
    <h1>Agregar Producto</h1>
    <form method="post" action="{% url 'add_producto' %}" enctype="multipart/form-data">
      {% csrf_token %}
      {{form|crispy}}
      <button type="submit" class="p-2 my-4 bg-blue-600 text-white hover:bg-blue-700 rounded">Guardar</button>
    </form>
{% endblock contenido %}

```

8. Editamos el archivo **catagolo/templates/producto/updare.html ,**

```
{% extends "base.html" %}
{% load tailwind_filters %}
{% block titulo %} Actualizar Producto{% endblock titulo %}
{% block contenido %}
    <h1>Actualizar Producto</h1>
    <form method="post" action="{% url 'update_producto' producto.id %}" enctype="multipart/form-data">
        {% csrf_token %}
        {{form|crispy}}
        <button type="submit" class="p-2 my-4 bg-blue-600 text-white hover:bg-blue-700 rounded">Guardar</button>
    </form>
{% endblock contenido %}

```

Dado de altas las url

9. Para dar de altas la url modificaremos el archivo **catalogo/urls.py** de la siguiente manera

```
from django.urls import path
from .views.categoria import lista_categorias, delete_categoria, add_categoria, update_categoria
from .views.producto import lista_productos, delete_producto, add_producto, update_producto



urlpatterns = [

    # Categorias
    path("categoria/", lista_categorias, name="list_categoria"),
    path("categoria/", lista_categorias, name="list_categoria"),
    path("categoria/agregar/", add_categoria, name="add_categoria"),
    path("categoria/actualizar/<int:id>/",
         update_categoria, name="update_categoria"),
    path("categoria/eliminar/<int:id>/",
         delete_categoria, name="delete_categoria"),
    # productos
    path("producto/", lista_productos, name="list_producto"),
    path("producto/agregar/", add_producto, name="add_producto"),
    path("producto/actualizar/<int:id>/",
         update_producto, name="update_producto"),
    path("producto/eliminar/<int:id>/",
         delete_producto, name="delete_producto"),
]

```

10. Para finalizar nuestro cruz deberamos de configurar la carpeta donde se guardaran la imágenes para ellos modificamos el archivo de **comercial/ settings.py** y agregamos al final del arcivo lo siguiente

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'

```

11. Ya estamos listos y lo unico que queda por realizar es realizar nuetras pruebas correspondientes

	**EJERCICIOS DE PRÁCTICA:**

Con todo lo anterio ya puede realziar todas las operaciones faltas de su sistema